

Analysis of Type-Driven approach to systems programming

Implementation of OpenGL library for Rust

Author: Mikołaj Depta

Supervisor: dr. Andrzej Łukaszewski

Date: 03.09.2024

Github: <https://github.com/mikiSpoko200/gpu-bulwark>

Agenda

- Introduction (5min)
 - Why Rust
 - Why OpenGL
 - Goals and key features
 - Overview of the project
- Code Examples (10min)
 - Live demo: gpu-bulwark vs raw OpenGL in C / C++
- Conclusions (5min)
 - What was achieved
 - What can be improved
 - Future plans

Introduction - Why Rust

Rust is a modern, statically typed, systems programming language. Goal of this study was to showcase that it can be successfully applied to graphics programming.

Notable features:

- state of the art type system
- natively compiled, with very good support for C interoperability
- precise semantics of resource management
- borrow checker which allows to better express how different pieces of program interact with each other.
- large ecosystem of open-source packages

Introduction - Why OpenGL

OpenGL is a very mature, cross-platform, well understood and widely supported graphics specification. It's also arguably the best API, for beginners in hardware accelerated graphics programming to start with.

However, it's also an old C API. Certain aspects of the programming interface make using OpenGL difficult despite its simple execution model.

The idea to create a Rust wrapper for OpenGL originated during the computer graphics course.

This is how the idea behind `gpu-bulwark` started.

Introduction - Goals

There were two main goals for this study.

1. Explore type-driven design in Rust

- Determine benefits and downsides of such an approach to systems programming
- Ascertain feasibility of developing software that way
- Identify common patterns for type-driven design

2. Create a minimalistic OpenGL wrapper

- Remain as close to original specification as possible
- Provide just the typing information and object oriented syntax
- Implement only the most essential functionality
- Prevent as many ill-formed programs at compile-time as possible

Introduction - `gpu-bulwark` key features

- Library implements the most essential parts of the OpenGL spec.
- Library statically detects misuse of the OpenGL API
- Successful compilation guarantees correct configuration of the graphics pipeline
- Extensible typing scheme for shaders, programs and vertices
- Type errors help to identify the cause of the problem

Examples

Shaders

Vertex shader

```
1 #version 420 core
2
3 layout(location = 0) in vec3 in_color;
4 layout(location = 1) in vec3 in_position;
5
6 layout(location = 0) out vec4 color;
7
8 void main() {
9     gl_Position = vec4(in_position, 1.0);
10    color = vec4(in_color, 1.0);
11 }
```

Fragment shader

```
1 #version 420 core
2
3
4 layout(location = 0) in vec4 color;
5 layout(location = 0) out vec4 out_color;
6
7
8 void main() {
9     out_color = color;
10 }
```

Vertex Shader

```
1 #version 420 core
2
3 layout(location = 0) in vec3 in_color;
4 layout(location = 1) in vec3 in_position;
5
6 layout(location = 0) out vec4 color;
7
8 void main() {
9     gl_Position = vec4(in_position, 1.0);
10    color = vec4(in_color, 1.0);
11 }
```

Listing 1 - Attribute size mismatch

Keeping track of shape of a N-dimensional arrays is a tedious and error prone process. Momentary lack of focus may cause invalid data to be provided to a routine. This will usually cause a run-time error, but since OpenGL exposes a C API and is provided as a binary for linking by the OS, it cannot use type information to bound check passed data.

Listing 1 - Attribute size mismatch

```
1 Buffer<float> colorBuffer = Buffer<float>::Array();
2 const std::vector<float> colors = {
3     1.0f, 0.0f, 0.0f, 1.0f,
4     0.0f, 1.0f, 0.0f, 1.0f,
5     0.0f, 0.0f, 1.0f, 1.0f,
6 };
7
8 colorBuffer.Data(colors, GL_STATIC_DRAW);
9 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT);
```

```
1 let glsl::vars! [ vin_color, vin_position ] = gb::glsl!
2     layout(location = 0) in vec3;
3     layout(location = 1) in vec3;
4 };
5 let mut colors = Buffer::create();
6
7 colors.data::<(Dynamic, Draw)>(&[
8     [1.0, 0.0, 0.0, 1.0],
9     [0.0, 1.0, 0.0, 1.0],
10    [0.0, 0.0, 1.0, 1.0f32]
11 ]);
12
13 let vao = VertexArray::create()
14     .vertex_attrib_pointer(&vin_color, colors);
```

Listing 2 - Attribute type mismatch

In this listing we show how using invalid constant value causes, one of the worst issues to diagnose: Blank screen.

Shader expects floating point data but we instead try providing integers. This causes OpenGL to bit-reinterpret integers as floats which results in incorrect behavior during rendering.

Listing 2 - Attribute type mismatch

```
1 Buffer<int> colorBuffer = Buffer<int>::Array();
2 const std::vector<int> colors = {
3     1, 0, 0,
4     0, 1, 0,
5     0, 0, 1,
6 };
7 colorBuffer.Data(colors, GL_STATIC_DRAW);
8 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT); // GL_FLOAT is not a
```

```
1 colors.data::<(Dynamic, Draw)>(&[
2     [1, 0, 0],
3     [0, 1, 0],
4     [0, 0, 1i32]
5 ]);
6 let vao = VertexArray::create()
7     .vertex_attrib_pointer(&vin_color, co
8     .vertex_attrib_pointer(&vin_position,
9     ;
```

Listing 3 - Specifying invalid attribute location

Another bug strong typing can detect, is using attribute index which does not match with vertex shader input variables.

Listing 3 - Specifying invalid attribute location

```
1 colorBuffer.Data(colors, GL_STATIC_DRAW);
2 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT);
3
4 positionBuffer.Data(positions, GL_STATIC_DRAW);
5 vao.VertexAttribPointer(0, positionBuffer, 3, GL_FLOAT);
```

```
1 let glsl::vars! [ vin_color, vin_position ] = gb::glsl! {
2     layout(location = 0) in vec3;
3     layout(location = 1) in vec3;
4 };
5
6 let vao = VertexArray::create()
7     .vertex_attrib_pointer(&vin_color, colors)
8     .vertex_attrib_pointer(&vin_color, positions)
9     ;
```

Listing 4 - Missing vertex attribute

Listing 4 - Missing vertex attribute

```
1 colorBuffer.Data(colors, GL_STATIC_DRAW);
2 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT);
3
4 positionBuffer.Data(positions, GL_STATIC_DRAW);
5 // vao.VertexAttribPointer(1, positionBuffer, 3, GL_FLOAT);
```

```
1 let vao = VertexArray::create()
2   .vertex_attrib_pointer(&vin_color, colors)
3   // .vertex_attrib_pointer(&vin_position, positions)
4   ;
```

Listing 5 - Letting compiler guide us

In the next few listings, we'll see how invalid configuration of vertex array can be detected at compile-time and fixed by following compilation error messages.

New Vertex Shader

```
1 #version 420 core
2
3 layout(location = 0) in ivec3 in_color;
4 layout(location = 1) in vec3 in_position;
5
6 layout(location = 0) out vec4 color;
7
8 void main() {
9     gl_Position = vec4(in_position, 1.0);
10    color = vec4(vec3(in_color / 16), 1.0);
11 }
```

Listing 5 - Letting compiler guide us

```
1 const std::vector<float> colors = {  
2     1.0f, 0.0f, 0.0f,  
3     0.0f, 1.0f, 0.0f,  
4     0.0f, 0.0f, 1.0f,  
5 };  
6  
7 colorBuffer.Data(colors, GL_STATIC_DRAW);  
8 vao.VertexAttribPointer(0, colorBuffer, 3, GL_FLOAT);
```

```
1 type VsInputs = glsl::Gls1! {  
2     layout(location = 0) in ivec3; // note `ivec`.  
3     layout(location = 1) in vec3;  
4 };  
5 type Attributes = gb::HList! {  
6     Attribute<f32; 3>, 0>, // but we provide f32s  
7     Attribute<f32; 3>, 1>,  
8 };  
9 let mut colors = Buffer::create();  
10  
11 colors.data::<(Dynamic, Draw)>(&[  
12     [1.0, 0.0, 0.0],  
13     [0.0, 1.0, 0.0],  
14     [0.0, 0.0, 1.0f32]  
15 ]);  
16
```

Listing 5 - Analyzing the error message

```
error[E0271]: type mismatch resolving `<[f32; 3] as FFI>::Layout == [i32; 3]`
--> src\listing.rs:127:34
127 |         self.program.draw_arrays(&self.vao);
    |         ^^^^^^^^^^^^^ expected `[i32; 3]`, found `[f32; 3]`
    |
    | required by a bound introduced by this call
    |
    = note: expected array `[i32; 3]`
    found array `[f32; 3]`
    = note: required for `[f32; 3]` to implement `gpu_bulwark::glsl::Compatible<GVec<i32, 3>>`
    = note: required for `(), Attribute<[f32; 3], 0>`
    to implement
```

Listing 5 - Analyzing the error message

From the error message, we can see that our vertex array provides incompatible type, for the first attribute, with the program input.

We dutifully check out attribute definitions and yes, there in fact is a bug in the first attribute.

```
1 type Attributes = gb::HList! {  
2   Attribute<[i32; 3], 0>, // we change attribute type to i32  
3   Attribute<[f32; 3], 1>,  
4 };
```


Listing 6 - Following error messages

```
error[E0308]: mismatched types
  --> src\listing.rs:108:13
108 |         vao,
    |         ^^^ expected `i32`, found `f32`
    = note: expected struct `VertexArray<(((), Attribute<[i32; 3], _>), Attribute<_, _>>)`
           found struct `VertexArray<(((), Attribute<[f32; 3], _>), Attribute<_, _>>)`
```

We follow error message to find all places affected by the original error.

```
1 let inner = Self {
2     program,
3     vao, // local variable vao is of incorrect type, now that we changed VA0's type
4 };
```

We can easily detect all other places in code that need fixing.

Listing 6 - Fixing attribute definition

Now, expectedly, we need to change the type of the constructed Vertex Array Object. Namely, the first attribute is ill-configured.

```
1 let vao = VertexArray::create()
2   .vertex_attrib_pointer(&vin_color, colors) // Buffer colors is to blame
3   .vertex_attrib_pointer(&vin_position, positions)
4   ;
```

We finally arrive at the root of the error - invalid contents of the colors buffer.

```
1 colors.data::<(Dynamic, Draw)>(&[
2   [1.0, 0.0, 0.0],
3   [0.0, 1.0, 0.0],
4   [0.0, 0.0, 1.0f32]
5 ]);
```

Listing 7 - Bug prevented

Having fixed all type errors by changing vertex data, we can run our sample. Which after compilation functions correctly.

```
1 colors.data::<(Dynamic, Draw)>(&[  
2     [15i32, 0, 0],  
3     [0, 15, 0],  
4     [0, 0, 15]  
5 ]);
```

New vertex shader

```
1 #version 460 core
2
3 layout(location = 0) in vec3 position;
4 layout(location = 1) in vec4 color;
5
6 layout(location = 0) uniform float multiplier;
7 layout(location = 1) uniform float x_offset;
8 layout(location = 2) uniform float y_offset;
9
10 layout(location = 0) out vec4 vertexColor;
11
12 void main() {
13     gl_Position = vec4(position, 1.0);
14     gl_Position.x += x_offset;
15     gl_Position.y += y_offset;
16     vertexColor = color * multiplier;
```

New vertex shader

```
1 #version 460 core
2
3 layout(location = 0) in vec3 position;
4 layout(location = 1) in vec4 color;
5
6 layout(location = 0) uniform float multiplier;
7 layout(location = 1) uniform float x_offset;
8 layout(location = 2) uniform float y_offset;
9
10 layout(location = 0) out vec4 vertexColor;
11
12 void main() {
13     gl_Position = vec4(position, 1.0);
14     gl_Position.x += x_offset;
15     gl_Position.y += y_offset;
16     vertexColor = color * multiplier;
```

New vertex shader

```
1 #version 460 core
2
3 layout(location = 0) in vec3 position;
4 layout(location = 1) in vec4 color;
5
6 layout(location = 0) uniform float multiplier;
7 layout(location = 1) uniform float x_offset;
8 layout(location = 2) uniform float y_offset;
9
10 layout(location = 0) out vec4 vertexColor;
11
12 void main() {
13     gl_Position = vec4(position, 1.0);
14     gl_Position.x += x_offset;
15     gl_Position.y += y_offset;
16     vertexColor = color * multiplier;
```

Listing 8 - Forgetting to set uniform location

New vertex shader expects 3 float uniforms.

```
layout(location = 0) uniform float multiplier;  
layout(location = 1) uniform float x_offset;  
layout(location = 2) uniform float y_offset;
```

In this sample we don't set uniform 2. Although, uniforms are zero-initialized during program linking it tends to cause even more trouble, since no error will be generated but rendering will usually produce nonsense; for example if the perspective matrix was zeroed.

```
glUniform1f(0, 1.0f);  
glUniform1f(1, 0.0f);  
// glUniform1f(2, 0.0f);
```

Listing 8 - Forgetting to set uniform location

In gpu-bulwark programs are configured by providing shaders for subsequent pipeline stages. These shaders include *declarations* of uniforms they use, which later must be matched against uniform *definitions* provided by the program.

```
1 type Uniforms = glsl::Gls! {  
2     layout(location = 0) uniform float;  
3     layout(location = 1) uniform float;  
4     layout(location = 2) uniform float;  
5 };  
6  
7 let glsl::vars! [ location_attenuation, location_x_offset, location_y_offset ] = Uniforms::default();  
8 let vs = uncompiled_vs  
9     .uniform(&location_attenuation)  
10    .uniform(&location_x_offset)  
11    .uniform(&location_y_offset)  
12    .compile()?  
13    .into_main()  
14    .inputs(&vs_inputs)  
15    .outputs(&vs_outputs);  
16 let fs = uncompiled_fs
```


Listing 8 - Forgetting to set uniform location

Below, from a valid program configuration one uniform definition has been removed.

```
1 let program = Program::builder()
2   .uniforms(|defs| defs
3     .define(&location_attenuation, &attenuation)
4     .define(&location_x_offset, &x_offset)
5     // .define(&location_y_offset, &y_offset)
6   )
7   .no_resources()
8   .vertex_main(&vs)
9   .uniforms(|matcher| matcher
10     // .bind(&location_y_offset)
11     .bind(&location_x_offset)
12     .bind(&location_attenuation)
13   )
14   .fragment_main(&fs)
15   .build()?;
```

Listing 9 - Uniform assignment overflow

In this sample, invalid function is used to set uniforms value. This code actually does not produce any runtime errors. If uniform being set incorrectly is followed by another uniform, invalid setting of the leading uniform will overflow and set the subsequent one. This is again, a bug which can be very difficult to diagnose.

Listing 9 - Uniform assignment overflow

```
1 glUniform1f(0, attenuation); CHECK_GL_ERROR;  
2 glUniform2f(1, 0.0f, 1.0f); CHECK_GL_ERROR; // We set uniform at location 1 as a vec2  
3 glUniform1f(2, y_offset); CHECK_GL_ERROR;
```

```
1 let glsl::vars! [ location_attenuation, location_x_offset, location_y_offset ] = gb::glsl! {  
2     layout(location = 0) uniform float;  
3     layout(location = 1) uniform float;  
4     layout(location = 2) uniform float;  
5 };  
6  
7 self.program.uniform(&location_attenuation, &self.attenuation);  
8 self.program.uniform(&location_x_offset, &[self.x_offset, self.y_offset]); // We set uniform at location 1 as a vec  
9 self.program.uniform(&location_y_offset, &self.y_offset);
```

Conclusions - Type-driven design

- The biggest benefit of extensive use of the type system is the static verification of the correctness of the analyzed programs
- Type-driven design forces greater attention during the initial phase of software creation
- The logical principles underpinning the type system allow for a very intuitive description of the program's structure, algorithm invariants, as well as the interactions between different components
- Enforcing greater attention during system architecture design brings significant benefits in later stages of development

Conclusions - Rust for OpenGL

- Interfacing with OS platform will be inherently unsafe
- Rust is a viable technology for programming computer graphics
- With initial overhead of providing safe facades around raw OS APIs, development becomes simpler than in C or C++
- Rust's type system can be used to express data formats to external APIs

What was achieved

- gpu-bulwark detects many kinds of OpenGL errors and unsound function invocations, at build time
- gpu-bulwark stays close to the OpenGL specification
- gpu-bulwark offers significant improvement over using OpenGL
- Type-driven design was shown to be beneficial for systems programming
- Many design patterns have been explored and documented

What can be improved

- Broader OpenGL API coverage
- Better source level documentation
- Separation of certain modules into crates
- Exploring alternative typing schemes to improve error messages
- Alternative compilation mode in which errors are checked at runtime
- Parsing of shader files to determine if glsl variable declarations match
- Alternative build target which prints glsl variable declarations to the console

Future plans - Integration with **rust-gpu**

```
use glam::{Vec3, Vec4, vec2, vec3};

#[spirv(fragment)]
pub fn main(
    #[spirv(frag_coord)] in_frag_coord: &Vec4,
    #[spirv(push_constant)] constants: &ShaderConstants,
    output: &mut Vec4,
) {
    let frag_coord = vec2(in_frag_coord.x, in_frag_coord.y);
    let mut uv = (frag_coord - 0.5 * vec2(constants.width as f32, constants.height as f32))
        / constants.height as f32;
    uv.y = -uv.y;

    let eye_pos = vec3(0.0, 0.0997, 0.2);
    let sun_pos = vec3(0.0, 75.0, -1000.0);
    let dir = get_ray_dir(uv, eye_pos, sun_pos);
```


Future plans - Publishing to `crates.io`

Source code for this project is already publicly available on github. Next step in proliferating type driven design is publishing this crate to `crates.io`.

One module of this library called `himark` was published during development, and as of today has over 700 unique downloads.

Future plans - Updating to use new language features

Due to some missing or incomplete language features certain parts of library use convoluted workarounds which heavily degrade error messages readability.

- With negative trait implementations we could cleanly implement checking that all locations are different
- With support for const generics in const expressions we could make the implementation of `ffi :: FFI` cleaner
- With support for implementation specification we could implement traits more abstractly
- With support for disjoint blanket implementations with equality bound on associated type we could avoid using very verbose workarounds

The End